

# Python Basics for CIVL6410

Jiwon Kim, The University of Queensland

2026

## Contents

|                                                                       |          |
|-----------------------------------------------------------------------|----------|
| <b>Python Basics for CIVL6410</b>                                     | <b>2</b> |
| 1. Variables and Basic Data Types . . . . .                           | 2        |
| Key points . . . . .                                                  | 2        |
| Try it yourself . . . . .                                             | 2        |
| 2. Lists . . . . .                                                    | 2        |
| Key points . . . . .                                                  | 3        |
| Try it yourself . . . . .                                             | 3        |
| 3. String Formatting with f-strings . . . . .                         | 3        |
| Key points . . . . .                                                  | 4        |
| Try it yourself . . . . .                                             | 4        |
| 4. Dictionaries . . . . .                                             | 4        |
| Key points . . . . .                                                  | 5        |
| Try it yourself . . . . .                                             | 5        |
| 5. Sets . . . . .                                                     | 5        |
| Key points . . . . .                                                  | 6        |
| 6. <code>if</code> / <code>elif</code> / <code>else</code> . . . . .  | 6        |
| Try it yourself . . . . .                                             | 6        |
| 7. <code>for</code> and <code>while</code> Loops . . . . .            | 6        |
| <code>for</code> loop — iterate over a known sequence . . . . .       | 6        |
| <code>while</code> loop — repeat until a condition is false . . . . . | 7        |
| <code>break</code> and <code>continue</code> . . . . .                | 7        |
| Try it yourself . . . . .                                             | 7        |
| 8. Functions ( <code>def</code> ) . . . . .                           | 8        |
| Key points . . . . .                                                  | 9        |
| Try it yourself . . . . .                                             | 9        |
| 9. Importing Modules . . . . .                                        | 9        |
| Quick Reference Summary . . . . .                                     | 9        |

# Python Basics for CIVL6410

## 1. Variables and Basic Data Types

A **variable** is a name that stores a value. Python has several built-in types.

```
# Integer (whole number)
age = 25
count = 100

# Float (decimal number)
price = 9.99
temperature = 36.6
max_val = float('inf')  # positive infinity - a very large number

# String (text)
filename = 'data.csv'
city = "Brisbane"

# Boolean (True / False)
is_open = False
has_error = True

# Check the type of a variable
print(type(age))          # <class 'int'>
print(type(price))        # <class 'float'>
print(type(city))         # <class 'str'>
print(type(is_open))      # <class 'bool'>
```

### Key points

- No need to declare the type — Python figures it out automatically.
- `float('inf')` represents positive infinity. You will see this used in algorithms to initialise a “best so far” value.
- Use `print()` to display values in the console.

### Try it yourself

1. Create a variable `speed` with value 60.0 and print its type.
2. Create a variable `country` with value "Australia" and print it.

## 2. Lists

A **list** stores an ordered sequence of items. Items can be of any type.

```
# Create a list
scores = [85, 92, 78, 90]
fruits = ['apple', 'banana', 'cherry']
result = []  # empty list

# Access by index (0-based)
```

```

print(scores[0])    # 85 (first element)
print(scores[-1])   # 90 (last element)

# Number of items
print(len(scores))  # 4

# Add to the end
scores.append(88)
print(scores)       # [85, 92, 78, 90, 88]

# Insert at a specific position
result.insert(0, 'c') # insert 'c' at index 0
result.insert(0, 'a') # insert 'a' at index 0
print(result)         # ['a', 'c']

# Remove a specific value
scores.remove(78)
print(scores)         # [85, 92, 90, 88]

# Check membership
print(92 in scores)   # True
print(78 in scores)   # False

# Iterate over a list
for s in scores:
    print(s)

```

## Key points

- Indices start at 0. The last element has index -1.
- `append` adds to the end; `insert(i, val)` adds before position `i`.
- `remove(val)` removes the first occurrence of `val`.
- `in` checks whether an item is present — very commonly used.

## Try it yourself

1. Create a list `todo = ['read', 'write', 'run']`. Remove `'write'` and print the result.
2. Check whether `'run'` is in `todo`.

## 3. String Formatting with f-strings

**f-strings** (formatted string literals) let you embed variable values directly inside a string.

```

name = 'Alice'
score = 92.5

# Basic f-string
print(f'{name} scored {score}')           # Alice scored 92.5

# Control decimal places
print(f'Score = {score:.1f}')             # Score = 92.5
print(f'Score = {score:.3f}')             # Score = 92.500

```

```

# Arithmetic inside {}
radius = 5
print(f'Area = {3.14159 * radius**2:.2f}')    # Area = 78.54

# Multiple variables
x, y = 3, 4
print(f'Point ({x}, {y}), distance from origin = {(x**2 + y**2)**0.5:.3f}')

```

## Key points

- Start the string with `f` before the opening quote.
- Put any Python expression inside `{ }`.
- Use `:.2f` to format a float to 2 decimal places, `:.0f` for no decimal places, etc.

## Try it yourself

1. Given `item = 'coffee'` and `price = 4.5`, print "coffee costs \$4.50" using an f-string.

## 4. Dictionaries

A **dictionary** stores key–value pairs. Think of it as a lookup table: given a key, you get a value.

```

# Create a dictionary
phone_book = {}                                # empty dictionary
phone_book['Alice'] = '0412 000 001'
phone_book['Bob']   = '0412 000 002'
phone_book['Carol'] = '0412 000 003'

print(phone_book)
# {'Alice': '0412 000 001', 'Bob': '0412 000 002', 'Carol': '0412 000 003'}

# Access a value by key
print(phone_book['Alice'])    # 0412 000 001

# Update a value
phone_book['Bob'] = '0412 999 999'
print(phone_book['Bob'])      # 0412 999 999

# Check if a key exists
print('Alice' in phone_book)  # True
print('Dave'  in phone_book)  # False

# Iterate over keys
for name in phone_book:
    print(name)

# Iterate over key-value pairs
for name, number in phone_book.items():
    print(f'{name}: {number}')

# Dictionary with integer keys

```

```
scores = {1: 85, 2: 92, 3: 78}
print(scores[2])    # 92

# Dictionary whose values are lists
class_groups = {}
class_groups['Group A'] = []
class_groups['Group A'].append('Alice')
class_groups['Group A'].append('Bob')
class_groups['Group B'] = ['Carol', 'Dave']
print(class_groups)
# {'Group A': ['Alice', 'Bob'], 'Group B': ['Carol', 'Dave']}
```

## Key points

- Keys can be strings, integers, or tuples (any *immutable* type).
- Values can be anything — numbers, strings, lists, even other dictionaries.
- Use `.items()` to iterate over both keys and values at once.

## Try it yourself

1. Create a dictionary `grades = {'Alice': 90, 'Bob': 75, 'Carol': 88}`. Update Bob's grade to 80 and print the dictionary.
2. Use a `for` loop to print every name and grade.

## 5. Sets

A **set** stores an unordered collection of *unique* items. Duplicate values are automatically removed.

```
# Create a set
colours = {'red', 'blue', 'green'}
print(colours)    # order may vary

# Add an element
colours.add('yellow')
print(colours)

# Duplicates are silently ignored
colours.add('red')
print(colours)    # still only one 'red'

# Check membership - fast and commonly used
print('blue' in colours)    # True
print('purple' in colours)  # False

# Remove an element (discard does not raise an error if item is absent)
colours.discard('green')
print(colours)

# Create a set from a list (removes duplicates)
numbers = [1, 2, 2, 3, 3, 3]
unique = set(numbers)
print(unique)    # {1, 2, 3}
```

## Key points

- Sets are unordered — you cannot access items by index.
- The main uses are fast membership testing (`in`) and deduplication.

## 6. `if` / `elif` / `else`

Conditional statements control which block of code runs based on a condition.

```
temperature = 36.6
```

```
if temperature >= 38.0:
    print("Fever - see a doctor")
elif temperature >= 37.0:
    print("Slightly elevated temperature")
else:
    print("Normal temperature")
```

Comparison operators: `==`, `!=`, `<`, `<=`, `>`, `>=`

Logical operators: `and`, `or`, `not`

```
# Combined condition
```

```
x = 5
```

```
if x > 0 and x < 10:
    print("x is a single digit positive number")
```

```
# Negation
```

```
found = False
```

```
if not found:
    print("Item not found")
```

```
# Checking membership inside a condition
```

```
fruits = ['apple', 'banana', 'cherry']
```

```
item = 'banana'
```

```
if item in fruits:
    print(f'{item} is in the list')
else:
    print(f'{item} is not in the list')
```

## Try it yourself

1. Write an `if/elif/else` block: if a variable `score` is `>= 85`, print "Distinction", if `>= 65` print "Credit", if `>= 50` print "Pass", otherwise print "Fail".

## 7. `for` and `while` Loops

`for` loop — iterate over a known sequence

```
fruits = ['apple', 'banana', 'cherry']
```

```
# Basic for loop
```

```

for item in fruits:
    print(item)

# Loop with range
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):   # 1, 2, 3, 4, 5
    print(i)

# Loop over a dictionary
scores = {'Alice': 90, 'Bob': 75, 'Carol': 88}
for name, score in scores.items():
    print(f'{name} scored {score}')

# Nested for loop - all combinations
rows = [1, 2, 3]
cols = ['A', 'B']
for r in rows:
    for c in cols:
        print(f'{c}{r}')    # A1, B1, A2, B2, A3, B3

```

**while loop — repeat until a condition is false**

```

todo = ['read', 'write', 'run']

while len(todo) > 0:
    current = todo[0]
    todo.remove(current)
    print(f'Done: {current}, remaining: {todo}')

```

**break and continue**

```

# break - exit the loop early
for i in range(1, 6):
    if i == 4:
        break
    print(i)    # prints 1, 2, 3

# continue - skip the current iteration
for i in range(1, 6):
    if i == 3:
        continue
    print(i)    # prints 1, 2, 4, 5

```

**Try it yourself**

1. Use a while loop to process queue = ['a', 'b', 'c'] one item at a time, printing each item as it is removed.
2. Use a nested for loop to print all (row, col) pairs where row is from [1, 2] and col is from ['X', 'Y', 'Z'].

## 8. Functions (def)

A **function** groups reusable code under a name. It can accept inputs (parameters) and return outputs.

```
# Basic function - no return value
def greet(name):
    print(f'Hello, {name}!')

greet('Alice')    # Hello, Alice!
greet('Bob')      # Hello, Bob!

# Function with a return value
def add(a, b):
    result = a + b
    return result

total = add(3, 5)
print(total)      # 8

# Function with a default parameter value
def make_greeting(name, language='English'):
    if language == 'English':
        return f'Hello, {name}!'
    elif language == 'French':
        return f'Bonjour, {name}!'
    else:
        return f'Hi, {name}!'

print(make_greeting('Alice'))          # Hello, Alice!    (uses default)
print(make_greeting('Alice', 'French')) # Bonjour, Alice! (overrides default)

# Function returning multiple values
def min_and_max(numbers):
    """Return the minimum and maximum values in a list.

    Parameters
    -----
    numbers : list    a list of numeric values

    Returns
    -----
    minimum : float    the smallest value
    maximum : float    the largest value
    """
    minimum = numbers[0]
    maximum = numbers[0]
    for n in numbers:
        if n < minimum:
            minimum = n
        if n > maximum:
            maximum = n
    return minimum, maximum

data = [4, 1, 7, 2, 9, 3]
lo, hi = min_and_max(data)
```



```
print(f'Min: {lo}, Max: {hi}')    # Min: 1, Max: 9
```

## Key points

- Using functions avoids copying the same code in multiple places. If you need to fix a bug, you fix it in one place.
- A function can return multiple values — they are returned as a tuple and unpacked by the caller (`lo, hi = ...`).

## Try it yourself

1. Write a function `count_above(values, threshold)` that takes a list of numbers and returns how many of them are greater than `threshold`.
2. Write a function `letter_grade(score)` that returns 'Distinction' for  $\geq 85$ , 'Credit' for  $\geq 65$ , 'Pass' for  $\geq 50$ , and 'Fail' otherwise.

## 9. Importing Modules

Python's standard library and third-party packages are loaded with `import`.

*# Import the whole module, then use the module name as a prefix*

```
import math
print(math.sqrt(16))    # 4.0
print(math.pi)          # 3.141592653589793
print(math.inf)         # inf
```

*# Import with a shorter alias*

```
import numpy as np
import pandas as pd
```

```
print(np.sqrt(16))      # 4.0
print(np.inf)           # inf
```

*# Import specific items from a module*

```
from math import sqrt, pi
print(sqrt(25))         # 5.0
print(pi)               # 3.141592653589793
```

## Quick Reference Summary

*# --- Data types ---*

```
x      = 5           # int
c      = 3.5         # float
name   = "Alice"     # str
flag   = True        # bool
```

*# --- List ---*

```
lst = [10, 20, 30]
lst.append(40)          # [10, 20, 30, 40]
lst.insert(0, 5)        # [5, 10, 20, 30, 40]
```

```

lst.remove(20)          # [5, 10, 30, 40]
print(30 in lst)        # True

# --- Dictionary ---
d = {}
d['a'] = 10
d['b'] = 20
for key, val in d.items():
    print(key, val)

# --- Set ---
s = {'red', 'blue', 'green'}
s.add('yellow')
print('blue' in s)      # True

# --- if/elif/else ---
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")

# --- for loop ---
for i in [1, 2, 3]:
    print(i)

# --- while loop ---
queue = ['a', 'b', 'c']
while len(queue) > 0:
    queue.remove(queue[0])

# --- function ---
def my_func(a, b):
    """Return the sum of a and b."""
    return a + b

# --- f-string ---
item, price = 'coffee', 4.5
print(f'{item} costs ${price:.2f}')

# --- import ---
import numpy as np
import pandas as pd

```